Atividade prática ATVIII

Professor Dr. Eng. Gerson Penha

Introdução:

Prezados empreendedores e empreendedoras, após termos consolidado a base da plataforma cinebridge com a aplicação de padrões criacionais e comportamentais, é chegada a hora de direcionarmos nossa atenção para um aspecto igualmente relevante, porém frequentemente negligenciado, da engenharia de software: a compreensão e o combate aos anti-padrões. A terceira etapa de nossa jornada nos convida a uma reflexão crítica sobre as más práticas que, embora pareçam soluções imediatas para problemas cotidianos, frequentemente se revelam armadilhas que comprometem a qualidade, a manutenibilidade e a longevidade de nossos sistemas. Esta introdução tem como propósito estabelecer os fundamentos teóricos e históricos dos anti-padrões, capacitando-os a identificar, evitar e, quando necessário, corrigir tais desvios em suas futuras empreitadas de desenvolvimento.

O conceito de anti-padrão emerge como uma resposta natural e quase imediata à sistematização dos padrões de projeto promovida pelo quarteto conhecido como Gang of Four, cuja obra seminal, publicada em 1994, consolidou uma linguagem comum para soluções arquiteturais confiáveis para problemas recorrentes no desenvolvimento orientado a objetos. Em 1995, o pesquisador e programador Andrew Koenig cunhou o termo "anti-padrão" em um artigo publicado no Journal of Object-Oriented Programming, inspirado precisamente pela necessidade de alertar a comunidade sobre o outro lado da moeda da reutilização de soluções. Koenig percebeu que muitos, senão todos, os padrões catalogados, quando não aplicados de maneira consciente e contextualizada, poderiam não apenas se tornar inefetivos, mas também representar riscos significativos aos projetos. Sua definição original é eloquente em sua simplicidade: um anti-padrão é uma solução que parece superficialmente uma solução, mas que na realidade não o é.

A popularização do termo e sua extensão para além da engenharia de software deve-se, em grande medida, à publicação do livro "AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis", em 1998, pelos autores William Brown, Raphael Malveau, Hays McCormick e Thomas Mowbray. Esta obra não apenas consolidou o conceito, mas também o expandiu para áreas como sociologia, administração e gestão de projetos, demonstrando que os anti-padrões são fenômenos recorrentes em qualquer atividade humana complexa que envolve processos, estruturas e interações sociais. Segundo a definição consagrada pelos autores, um anti-padrão é uma prática comum em arquitetura de software, porém inefetiva e contraproducente para um problema recorrente, causando, de forma geral, mais problemas do que benefícios. Para que uma má prática seja elevada à categoria de anti-padrão, dois critérios fundamentais precisam ser atendidos: primeiro, deve ser uma ação, processo ou estrutura que se repete e que, a princípio, parece benéfica, mas que se revela, ao final, contraproducente; segundo, deve existir um processo de refatoração bem documentado e comprovadamente eficaz para corrigir a situação.

A identificação de um anti-padrão em um sistema não é uma tarefa trivial, pois exige do engenheiro de software uma combinação de experiência técnica, discernimento arquitetural e sensibilidade para os sinais de deterioração estrutural. Os anti-padrões manifestam-se sob diversas formas, desde a proliferação de código duplicado até a criação de hierarquias de herança excessivamente profundas e frágeis. Alguns dos anti-padrões mais conhecidos e estudados incluem o "Blob" ou "Objeto Deus", onde uma única classe assume responsabilidades

excessivas, tornando-se o ponto central de dependência e fragilidade do sistema; a "Herança Profunda", que cria cadeias de dependência quebradiças e difíceis de modificar; e o "Spaghetti Code", caracterizado por uma estrutura de controle emaranhada e sem modularidade, que torna o código praticamente ilegível e impossível de manter. A identificação bem-sucedida de um anti-padrão requer, portanto, uma postura vigilante e crítica durante o processo de desenvolvimento e revisão de código, bem como a utilização de métricas de qualidade de software que possam apontar áreas de risco.

Uma questão particularmente sutil e de grande relevância prática para os empreendedores é saber se um padrão de projeto consagrado pode, em determinadas circunstâncias, transformar-se em um anti-padrão. A resposta, infelizmente, é afirmativa. Um padrão de projeto não é uma receita infalível, mas sim uma solução para um problema em um contexto específico. A aplicação de um padrão fora de seu contexto apropriado ou sem uma compreensão adequada de suas consequências pode gerar complexidade desnecessária, sobrecarga de desempenho e rigidez estrutural, exatamente o oposto do que se pretende alcançar. Um exemplo clássico é o padrão Singleton, que garante a existência de uma única instância de uma classe em toda a aplicação. Embora útil para acessar recursos compartilhados, como um arquivo de configuração, seu uso indiscriminado pode transformar o Singleton em um "Global State" disfarçado, criando dependências ocultas e dificultando a testabilidade do sistema. Da mesma forma, o padrão Factory Method, que estudamos na atividade anterior, pode se tornar um anti-padrão se for utilizado para criar objetos que não compartilham uma interface comum ou se a hierarquia de fábricas crescer a ponto de se tornar incontrolável. A lição fundamental é que a qualidade de uma solução não reside apenas no padrão escolhido, mas na sabedoria do engenheiro em discernir quando, como e por que aplicá-lo, sempre ponderando os benefícios contra os custos de complexidade introduzidos. A terceira atividade, que ora se inicia, consistirá justamente em uma análise aprofundada do código da plataforma cinebridge, com o objetivo de identificar possíveis anti-padrões que tenham sido introduzidos inadvertidamente e, em seguida, propor refatorações baseadas em padrões de projeto bem fundamentados para eliminar esses desvios, transformando um sistema que funciona em um sistema que realmente merece a confiança de seus usuários e mantenedores.

Atividade:

Prezados empreendedores e empreendedoras, após a profunda reflexão sobre os anti-padrões e sua identificação em sistemas de software, é chegado o momento de aplicarmos esse conhecimento na prática, transformando a plataforma cinebridge em um sistema ainda mais robusto, flexível e preparado para o crescimento. A terceira atividade consistirá em um ciclo de refatoração e extensão do microsserviço de recomendação e orquestração, com o objetivo específico de introduzir os padrões Adapter, Composite e Decorator. Estes padrões, de natureza estrutural, abordam problemas fundamentais relacionados à integração de componentes heterogêneos, à representação de estruturas hierárquicas e ao enriquecimento dinâmico de funcionalidades sem a necessidade de modificar as classes existentes. A aplicação destes padrões responderá a requisitos emergentes da plataforma, que agora precisa se conectar a sistemas externos de pagamento e armazenamento, gerenciar pacotes de serviços compostos e adicionar capacidades de auditoria e cache de forma transparente.

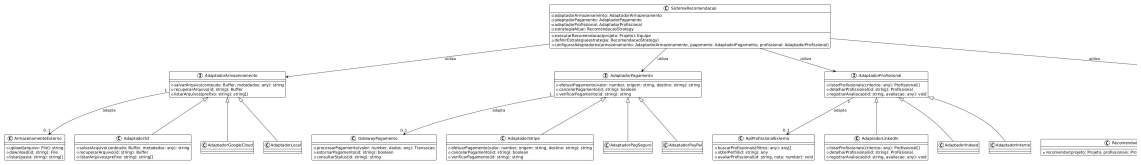
O contexto que motiva esta refatoração é o seguinte. Até o momento, nosso microsserviço operava de forma relativamente isolada, consumindo e produzindo dados internamente, sem interações significativas com o ecossistema externo. Contudo, a experiência prática no setor audiovisual independente revela que a plataforma cinebridge precisa se integrar a uma

variedade de serviços de terceiros. Em primeiro lugar, é imperativo que o sistema se conecte a provedores de armazenamento em nuvem, como Amazon S3 ou Google Cloud Storage, para gerenciar os arquivos de mídia associados aos projetos, tais como roteiros, storyboards e amostras de vídeo. Em segundo lugar, a plataforma precisa se integrar a gateways de pagamento, como Stripe ou PagSeguro, para processar as transações financeiras entre produtores e profissionais. Em terceiro lugar, o sistema deve ser capaz de consumir dados de diferentes fontes de cadastro de profissionais, incluindo APIs de redes sociais profissionais e portais de emprego especializados. Cada um destes sistemas externos possui sua própria interface, seus próprios formatos de dados e seus próprios protocolos de comunicação, o que torna a integração direta um desafio de manutenção e extensibilidade. O padrão Adapter surge como a solução adequada para este problema, encapsulando a lógica de comunicação com cada serviço externo em adaptadores que expõem uma interface padronizada para o núcleo do sistema, isolando assim a lógica de negócio das peculiaridades técnicas de cada provedor.

Paralelamente, os requisitos de negócio evoluíram para incluir a noção de pacotes de serviços compostos. Um projeto audiovisual não é mais visto como uma simples lista de papéis técnicos, mas sim como uma hierarquia de serviços, onde um pacote pode conter subpacotes e serviços atômicos. Por exemplo, um pacote de pós-produção pode incluir serviços de edição, coloração, mixagem de áudio e efeitos visuais, cada um dos quais pode, por sua vez, ser decomposto em subserviços mais específicos. Além disso, os produtores podem criar pacotes customizados, combinando serviços de diferentes categorias. A representação e manipulação desta estrutura hierárquica, onde cada elemento pode ser tratado de maneira uniforme, seja ele um serviço atômico ou um composto, é o problema endereçado pelo padrão Composite. Este padrão permitirá que o sistema calcule o custo total de um pacote, avalie a disponibilidade de todos os seus componentes e aplique recomendações de forma recursiva, tratando cada nó da hierarquia com a mesma interface, independentemente de sua complexidade interna. A implementação deste padrão tornará a lógica de orquestração mais limpa e extensível, permitindo a adição de novos tipos de serviços sem a necessidade de modificar os algoritmos existentes.

Por fim, a plataforma cinebridge requer a adição de funcionalidades transversais a todos os componentes do sistema de recomendação, tais como registro de auditoria detalhado, cache de resultados de recomendações para projetos similares e compressão de dados para reduzir o tráfego de rede. Estas funcionalidades não fazem parte da responsabilidade principal do orquestrador ou das estratégias de recomendação, mas sim se sobrepõem a elas. O padrão Decorator oferece um mecanismo elegante para adicionar tais comportamentos de forma dinâmica e composicional, envolvendo os objetos centrais em camadas de responsabilidades adicionais. Por exemplo, um recomendador que implementa a lógica de similaridade de cosseno pode ser decorado com um cache que verifica se a mesma requisição já foi processada recentemente, e então decorado novamente com um auditador que registra cada chamada e seu tempo de resposta. Esta abordagem evita a proliferação de subclasses para cada combinação de funcionalidades e mantém o princípio de responsabilidade única, permitindo que cada decorador se concentre em uma única preocupação transversal.

Para orientar os empreendedores nesta empreitada, apresento o diagrama de classes revisado, com todas as cardinalidades e tipos explicitamente definidos. Este diagrama incorpora os três novos padrões e serve como guia arquitetural para as alterações a serem implementadas.



A proposta de trabalho para esta terceira atividade é a seguinte. Os empreendedores deverão, a partir do código existente do microsserviço de recomendação, realizar as seguintes tarefas. Em primeiro lugar, definir as interfaces AdaptadorArmazenamento, AdaptadorPagamento e AdaptadorProfissional, que estabelecerão os contratos padronizados para comunicação com os sistemas externos. Em seguida, implementar pelo menos duas classes concretas para cada interface, representando diferentes provedores, como AdaptadorS3 e AdaptadorGoogleCloud para armazenamento, AdaptadorStripe e AdaptadorPagSeguro para pagamentos, e AdaptadorLinkedIn e AdaptadorInterno para cadastro de profissionais. Os empreendedores deverão criar as classes que representam os sistemas externos simulados, ou seja, classes que imitam o comportamento de APIs reais, e então implementar os adaptadores que traduzem as chamadas padronizadas para as chamadas específicas de cada sistema. O SistemaRecomendacao deve ser modificado para receber, em seu construtor ou por meio de métodos de configuração, as instâncias de cada adaptador, de modo que a lógica de negócio não dependa mais de implementações concretas.

Em segundo lugar, os empreendedores deverão criar a hierarquia de serviços para o padrão Composite. A interface ServicoProjeto deve ser definida com os métodos calcularCusto, listarSubservicos, obterPrazo e validar. A classe ServicoAtomico deve implementar esta interface para representar serviços individuais, como edição de vídeo ou mixagem de áudio, cada um com seu custo, prazo e papel técnico associado. A classe PacoteServico, por sua vez, deve implementar a mesma interface, mas com a capacidade de conter uma lista de ServicoProjeto, podendo ser atômico ou composto. Os empreendedores devem modificar a classe Projeto para que ela passe a ter um atributo servicoPrincipal do tipo ServicoProjeto, em vez de uma lista simples de papéis. O método calcularCustoTotal do projeto deve simplesmente delegar a operação ao serviço principal, que, por sua vez, a propagará recursivamente por toda a hierarquia. A validação da estrutura do projeto também deve ser recursiva, garantindo que todos os serviços atômicos tenham papéis válidos e que os prazos sejam consistentes.

Em terceiro lugar, os empreendedores deverão implementar o padrão Decorator para adicionar funcionalidades transversais ao mecanismo de recomendação. A interface RecomendacaoStrategy permanece inalterada. A classe RecomendadorBase deve conter a lógica principal de recomendação, como já implementada na primeira atividade. As classes CacheDecorator, AuditoriaDecorator e CompressaoDecorator devem implementar a mesma interface e receber, em seus construtores, uma instância de RecomendacaoStrategy que será envolvida. No método recomendar, cada decorator deve executar sua lógica adicional, como verificar o cache, registrar logs ou comprimir os dados de saída, e então delegar a chamada ao recomendador interno. Os empreendedores devem demonstrar que é possível combinar múltiplos decoradores em diferentes ordens, como primeiro aplicar o cache, depois a auditoria e, finalmente, a compressão, sem que a classe SistemaRecomendacao precise ser modificada. O sistema deve ser configurado para permitir que a pilha de decoradores seja definida dinamicamente em tempo de execução, com base em políticas de configuração ou preferências do produtor.

Finalmente, os empreendedores deverão implementar testes de integração abrangentes que demonstrem o funcionamento correto de cada um dos novos padrões. Para o Adapter, os testes devem mostrar que o sistema pode trocar de provedor de armazenamento ou pagamento sem alterar a lógica de negócio, simplesmente substituindo a instância do adaptador injetada no SistemaRecomendacao. Para o Composite, os testes devem validar que o cálculo de custos e prazos funciona corretamente para estruturas de serviços de diferentes profundidades e complexidades. Para o Decorator, os testes devem confirmar que a adição de cache reduz o tempo de resposta para requisições repetidas, que a auditoria registra corretamente todas as chamadas e que a compressão reduz o tamanho dos dados transmitidos, tudo isso sem que o recomendador base precise ser alterado. Ao final desta atividade, os empreendedores terão um microsserviço não apenas funcional, mas também altamente adaptável a mudanças nos serviços externos, capaz de representar hierarquias complexas de serviços e de adicionar funcionalidades transversais de forma modular e não intrusiva. Aceitem este desafio com a convicção de que a engenharia de software, em sua essência, é a arte de gerenciar a complexidade, e que cada padrão de projeto é uma ferramenta valiosa neste ofício.

Por fim, para que o sistema possa ser comercializado com facilidade ele de ser compatível com as plataformas Windows 10 ou superior, Linux Ubuntu 24.04.03 ou superior e distribuições Linux derivadas do Ubuntu.